

Reproducing the Temeraire Hugepage-Aware Allocator Results in a Containerized Single-Host Environment

Aldo Costa

aldo.costa@hhu.de

Heinrich-Heine-Universität

Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire is a hugepage-aware extension of TCMalloc that aims to improve application performance by preserving hugepage coverage and reducing TLB pressure. The original OSDI 2021 paper evaluates Temeraire through application studies, a Google fleet experiment, and a production rollout. This report deliberately narrows reproduction to the part that can be approximated with public code: the Redis 6.0.9 case study.

The artifact evolved from a generic allocator benchmark into a Redis-focused reproduction by pinning Redis, selecting a historical public TCMalloc revision, building separate legacy and hugepage-aware shared libraries, verifying allocator selection through LD_PRELOAD, and documenting THP behavior during the benchmark. Several practical issues shaped the final setup, including public-code divergence from Google’s internal artifact, wrapper visibility constraints, exact LLVM/clang matching, WSL2/Docker THP behavior, and a post-audit release-on configuration error.

With THP active and periodic release disabled, the local median Temeraire advantage is +0.48%, compared with the paper’s +0.75%. After correcting the release-on setup, a 24-run sensitivity matrix tested 16, 64, and 256 MiB/s. The rate medians were −0.21%, +6.80%, and +0.20%, but the large middle value coincided with severe host-throughput drift. The result is a public reconstruction that exposes both the allocator signal and the limits of consecutive hour-long measurements on Docker/WSL2.

1 Introduction

Memory allocation is often treated as a local implementation detail of C and C++ programs. Temeraire starts from a different observation: allocator decisions shape where objects land in memory, whether huge pages remain usable, and how much address-translation work the processor must perform later. Hunter et al. therefore connect allocator design to fleet efficiency as well as the direct cost of malloc and free [1].

A full reproduction of the Temeraire paper is outside the scope of this seminar artifact. The paper’s main evidence comes from Google’s production environment, including internal workloads, large-scale telemetry, and fleet-level rollout data. Those conditions are not available externally. This report therefore focuses on the Redis 6.0.9 case study: a public workload where the paper gives enough benchmark detail to support a local reproduction attempt.

The goal is not to reconstruct Google’s internal allocator binary. Instead, the artifact asks a narrower question: can the Redis comparison be approximated with public code closely enough to observe the same small-effect regime? That question required more than

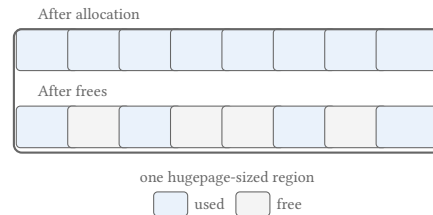


Figure 1: Fragmentation inside a hugepage-sized region. Free memory exists, but live objects prevent the whole region from being released intact. Redrawn after Figure 1 in Hunter et al. [1].

running a benchmark. The repository had to be reshaped, the allocator comparison had to be reconstructed, and the local hugepage path had to be verified explicitly.

2 Background and Temeraire Mechanism

Processors translate virtual addresses to physical addresses through page tables. Because page-table walks are expensive, recent translations are cached in translation lookaside buffers (TLBs). With ordinary 4 KiB pages, large working sets can require many translations. A 2 MiB hugepage covers the same address range as 512 ordinary pages, so one hugepage TLB entry can cover much more memory than one ordinary-page entry.

The benefit is not that 2 MiB of data are loaded into cache at once. The benefit is that fewer cached address translations are needed. Whether that benefit is realized depends on layout: hugepage mappings require suitably aligned and contiguous regions. Linux Transparent Huge Pages (THP) can create such mappings, but its success depends partly on how the user-space allocator arranges live objects.

Figure 1 shows the central trade-off. Returning small free pieces to the OS can reduce resident memory, but it may split hugepage mappings. Keeping the hugepage intact preserves TLB coverage, but leaves some physical memory idle. Temeraire exists to make this trade-off explicit inside the allocator backend.

TCMalloc is organized as a hierarchy of caches. Many small allocations are served by upper per-CPU, transfer, or central-cache layers. Larger page-granularity requests eventually reach the page-heap, which obtains memory from the OS and releases unused spans. Temeraire modifies this backend pageheap layer rather than replacing the whole allocator.

Figure 2 matters for the reproduction because Temeraire is not tested as a completely separate allocator. The experiment compares

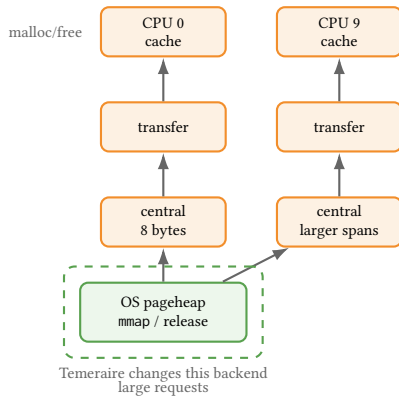


Figure 2: Simplified TCMalloc hierarchy. Temeraire changes the backend pageheap layer while leaving the upper cache structure intact. Redrawn after Figure 3 in Hunter et al. [1].

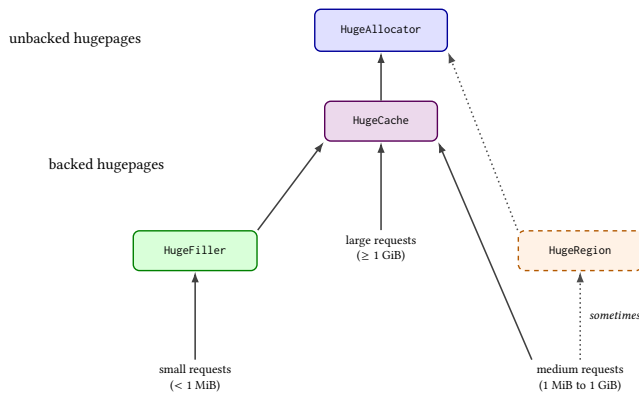


Figure 3: Temeraire's component structure. Small requests are routed to HugeFiller, large requests to HugeCache, and medium requests usually to HugeCache but occasionally to HugeRegion. Adapted after Figure 6 in Hunter et al. [1].

two backend configurations of TCMalloc: the legacy pageheap and the hugepage-aware path.

Figure 3 summarizes the backend components relevant for the artifact. **HugeAllocator** manages hugepage-aligned virtual address ranges. **HugeCache** keeps completely empty backed hugepages for reuse. **HugeFiller** handles small allocations and is the key component for Redis. **HugeRegion** handles medium-sized allocations that would otherwise create large slack regions.

The key Redis-relevant policy is in **HugeFiller**. For a request of size K , it prefers a hugepage whose longest free run is just large enough for the request. When several candidates are equivalent, it prefers the fuller one. This avoids consuming long free runs unnecessarily and avoids disturbing nearly empty hugepages that might otherwise drain and be released.

The Redis list workload in the paper performs small list-node and string-value allocations. Under Temeraire, these requests are expected to pass through **HugeFiller**; under the legacy pageheap,

they are handled without the same hugepage-aware placement objective. The experimental hypothesis is therefore small and specific: if Temeraire consolidates Redis allocations more effectively, Redis should show a small throughput improvement from reduced TLB pressure.

Periodic memory release complicates this effect. When release is enabled, TCMalloc can return memory to the OS using mechanisms such as `MADV_DONTNEED`. This can interact with hugepage coverage and add variance. For this reason, the report treats release-off as the cleaner placement signal and release-on as a noisier condition.

3 Reproduction Target and Artifact Evolution

The original paper reports application studies, a fleet experiment, and a production rollout. Only the Redis case study is realistic for a public seminar reproduction. The paper gives Redis 6.0.9, a legacy TCMalloc pageheap baseline, 2000 redis-benchmark trials, 1,000,000 requests per trial, and a workload combining a five-element push with a five-element read [1]. It also states the CPU family and LLVM commit used for compilation. It does not specify the exact Linux distribution, kernel version, or THP policy for the Redis run. Those properties can therefore be documented locally, but not matched exactly.

The repository did not initially match this target. It was a useful allocator benchmark scaffold, but its defaults used Redis 7.2.5, compared glibc against open-source gperftools TCMalloc, and did not expose a clean legacy-pageheap versus Temeraire comparison. The first reproduction step was therefore to narrow and correct the claim: the artifact would reproduce the Redis case-study methodology only, not the paper's full evaluation.

The paper's internal allocator build is not available. The artifact therefore uses a historical public google/tcmalloc commit, 8e534f507074, because it still exposes the `want_no_hpa` mechanism needed to disable the hugepage-aware path. This makes a public legacy-versus-Temeraire split possible, but it should not be read as byte-identical to Google's internal artifact.

The public repository also does not provide ready-made shared libraries for Redis preloading. The setup script therefore adds a small wrapper target inside the cloned TCMalloc tree and builds two loadable variants. The Temeraire build links against `//tcmalloc`; the legacy build also links against `//tcmalloc:want_no_hpa`. The benchmark harness selects the desired library through `LD_PRELOAD`.

Bazel was helpful because it let the artifact use TCMalloc's native build graph instead of reconstructing allocator dependencies, compiler flags, and link semantics in a separate build system. It also kept the comparison narrow: both shared libraries are produced from the same wrapper source, and the intended difference is the allocator target plus the extra `//tcmalloc:want_no_hpa` dependency in the legacy build. The cost was integration friction. Bazelisk had to be installed and pinned, the exact LLVM compiler path had to be passed through Bazel's action and repository environments, an external wrapper workspace did not inherit the right dependencies, and target visibility forced the wrapper into `tcmalloc/testing`.

Runtime verification is necessary. Redis can still report `mem_allocator:libc` from its own build metadata even when the process allocator was replaced through `LD_PRELOAD`. The artifact therefore checks `/proc/$pid/maps` to confirm that the intended shared library is actually mapped into the Redis process.

The final artifact reflects several failed or revised approaches. A modern `google/tcmalloc` checkout was not a convenient starting point for this wrapper strategy, and an external Bazel wrapper workspace did not naturally inherit all TCMalloc dependencies such as Abseil. The successful path was to add the wrapper inside the cloned TCMalloc source tree. Visibility was another obstacle: the `want_no_hpa` target could not be linked from an arbitrary external package, so the wrapper had to be placed where TCMalloc's own visibility rules permitted the dependency.

A later validation run uncovered a symbol mismatch. The wrapper initially referenced background-release methods that were not present in the pinned historical TCMalloc revision. The shared objects built, but failed to load before Redis started. The fix was to resolve those methods dynamically when available, allowing the wrapper to load against the historical revision while preserving the intended behavior for newer revisions.

Compiler matching also proved more expensive than it first looked. The paper mentions LLVM commit `cd442157cf` and `-O3`. Matching that statement required a local LLVM/clang toolchain, then using it for Redis, gperftools, and the Bazel-based TCMalloc wrapper build. Later metadata confirmed use of a local LLVM 13.0.0 build from the longer `cd442157cf` commit. One older manifest still marked the exact LLVM path as disabled, so the compiler metadata is treated as authoritative.

The most important methodological difficulty was THP. Initial long runs under WSL2/Docker with THP in `madvise` mode recorded `AnonHugePages:0kB` for Redis. These runs looked like controlled allocator comparisons, but they did not provide evidence that the hugepage mechanism was active. The harness was therefore extended to record THP settings and periodic `smaps_rollup` snapshots, and the final paper-closer runs set THP to `always` before benchmarking.

4 Methodology

The final workflow has four stages: build the container, set up pinned sources, verify allocator preload, and run the paper-closer Redis benchmark.

The artifact contains two benchmark paths. The direct Redis script is mainly useful for smoke tests, allocator-preload validation, and exploratory local measurements. The final reported results use the paper-closer script. That orchestration fixes the Redis workload to the paper-shaped configuration, separates release-off and release-on runs, records known deviations and allocator order, supports balanced reruns, and runs after THP is set to `always`. Thus, the direct path validates that the artifact works, while the paper-closer path defines the runs used for comparison with the paper.

The container image is based on `debian:bookworm-slim`, so the documented user-space Linux distribution for the experiment is Debian 12/bookworm. This does not pin the active Linux kernel. Under Docker Desktop with WSL2, Redis runs inside the Debian container while using the shared Docker/WSL Linux kernel. Kernel release,

THP behavior, cgroup state, NUMA topology, and hardware-counter availability are therefore recorded as experimental metadata rather than treated as properties fixed by the Dockerfile.

The latest recorded system snapshot used for the reported artifact metadata is `results/raw/system-info/20260524T094418Z.txt`. It records Debian GNU/Linux 12 (bookworm) user space on the WSL2 kernel `6.6.114.1-microsoft-standard-WSL2`, built on Dec. 1, 2025, for `x86_64`. The same snapshot records THP and THP defrag with `always` selected, and `khugepaged/max_ptes_none=511`. Since the Docker image base did not change, this snapshot is representative of the container user-space version used by the reported runs, while the kernel fields remain host/WSL2 environment metadata.

In compact form, the working command sequence is:

```
docker compose build
docker compose run --rm temeraire-dev bash -lc "./scripts/
  setup_env.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/
  check_allocator_preload.sh"
docker compose run --rm temeraire-dev bash -lc "echo always
  > /sys/kernel/mm/transparent_hugepage/enabled && echo
  always > /sys/kernel/mm/transparent_hugepage/defrag"
docker compose run --rm temeraire-dev bash -lc "./scripts/
  run_paper_closer_redis_experiment.sh"
```

The paper-closer script runs Redis 6.0.9 with 2000 trials, 1,000,000 requests per trial, 50 clients, and pipeline depth 16. Each trial flushes the database and then benchmarks two operations:

- `LPUSHbenchmark: listv1v2v3v4v5;`
- `LRANGEbenchmark: list04.`

The script records software versions, CPU information, kernel and OS metadata, THP settings, Redis version, TCMalloc commit, compiler metadata, allocator artifacts, raw benchmark output, and memory snapshots. Balanced reruns alternate which allocator runs first so that a sub-one-percent signal is less likely to be confused with fixed-order host drift.

The metadata collection distinguishes the container OS from the active kernel: it records the Debian container release, Debian version, Docker/container markers, shared kernel release and version, cgroup context, CPU model, memory summary, THP settings, Redis version, TCMalloc commit, compiler versions, and allocator artifacts. This distinction matters because the experiment used Debian bookworm user space, but hugepage behavior came from the shared Linux kernel visible to Docker/WSL2.

The primary metric is throughput in requests per second from `redis-benchmark`. `LPUSH` and `LRANGE` are collapsed into a combined push-plus-read rate using the harmonic mean

$$\frac{2}{1/r_{LPUSH} + 1/r_{LRANGE}}.$$

This treats the two throughputs as rates for sequential operations and keeps the local comparison structurally aligned with the paper's single Redis delta per release mode. The harmonic-mean aggregation itself is not specified in the paper. It is a local reconstruction choice because our public `redis-benchmark` harness records `LPUSH` and `LRANGE` as separate operation means, while the paper reports one Redis throughput delta for the combined push-five/read-five workload.

The hypothesis is deliberately modest. Redis creates a stream of small allocations. Under Temeraire, these allocations are subject to `HugeFiller`'s placement heuristic; under the legacy pageheap, they are not. If Temeraire improves hugepage coverage for this workload, Redis should show a small positive throughput delta. The effect is expected to be clearest with periodic release disabled, because release-on adds interaction with background memory return and can disrupt or mask the placement signal.

4.1 Post-audit correction to the release-on condition

The initial analysis treated runs labelled `paper-release-on` as periodic release experiments. A later code audit showed that this label was wrong. The runner started TCMalloc's background-actions thread but did not set its background release rate. The pinned public TCMalloc revision initializes that rate to zero. Its background loop therefore continued per-CPU-cache maintenance, but calculated zero bytes for the pageheap release call.

The existing release-on logs remain part of the experimental record, including the -12.02% run and its targeted rerun. They do not measure periodic pageheap release and cannot be compared with the paper's $+0.44\%$ Redis result. A corrected experiment must set and record a positive release rate. Since the paper does not publish that rate, the chosen value will remain an explicit local assumption and should be tested across several rates.

The correction itself became another part of the reproduction work. The first smoke test showed that the wrapper's dynamic symbol lookups had been failing silently, so the wrapper was changed to call the linked TCMalloc API directly. It also revealed that `LD\_PRELOAD` had leaked from the Redis server to `redis-cli` and `redis-benchmark`; the preload is now scoped to the server process. Rebuilding required one more Bazel dependency fix, followed by Linux line-ending cleanup for the container scripts. The corrected run was only started after both allocator smoke tests logged the requested positive rate.

The completed sensitivity matrix contains four alternating-order allocator pairs at each of 16, 64, and 256 MiB/s, for 24 allocator runs in total. Every run contains 2000 trials and an explicit server-log confirmation of its rate.

5 Results

The release-off analysis uses runs produced through the paper-closer workflow. Earlier direct-path runs are treated as environment validation and workflow development artifacts. The misconfigured release-on runs appear only to retain the audit trail; they are not paper-comparable measurements. Likewise, earlier paper-closer attempts with `AnonHugePages: 0kB` are useful workflow validation, but not evidence for the hugepage mechanism. After THP was set to always, the reported runs showed non-zero `AnonHugePages` in Redis snapshots.

Table 1 shows the valid release-off comparisons and preserves the invalid release-on values for traceability. Four of five release-off runs favor Temeraire, and the median delta is $+0.48\%$, compared with the paper's $+0.75\%$ Redis[†] result. The local pairs vary from -0.76% to $+3.46\%$, so this is a descriptive small-effect signal rather than a precise reproduction of the paper's value.

Table 1: Run-level Temeraire throughput deltas in THP-enabled runs. L first and T first denote allocator order. Off is the valid release-disabled condition. The on column is retained as a record of the misconfigured zero-release-rate runs and is not a paper-comparable result.

Run	Order	Off	On
fixed-order	L first	$+1.88\%$	$+0.26\%$
balanced 1	L first	$+0.43\%$	$+0.12\%$
balanced 2	T first	$+0.48\%$	$+0.38\%$
balanced 3	L first	-0.76%	-2.34%
balanced 4	T first	$+3.46\%$	-12.02%
targeted on	T first	–	$+0.12\%$

The release-on column is not interpreted as an allocator result. Balanced rerun 4 produced a -12.02% delta, and the targeted rerun returned $+0.12\%$, but both ran with a zero pageheap release rate. These values describe the misconfigured condition only; the targeted rerun does not repair the missing release-rate control.

Table 2: Paper comparison and local interpretation.

Mode	Paper	Local summary	Interpretation
off	$+0.75\%$	median $+0.48\%$	small, highly variable signal
on	$+0.44\%$	rate sweep completed	host drift dominates

Table 3: Corrected release-on sensitivity results. Each cell is the Temeraire throughput delta against its paired legacy run.

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Median
16 MiB/s	$+1.91\%$	-0.16%	-0.27%	-1.37%	-0.21%
64 MiB/s	-6.88%	$+2.10\%$	$+11.49\%$	$+12.21\%$	$+6.80\%$
256 MiB/s	$+15.73\%$	-0.09%	$+0.50\%$	-0.77%	$+0.20\%$

The sensitivity results are not monotonic. The 16 MiB/s pairs are centered near zero, as are three of four 256 MiB/s pairs. The 64 MiB/s range spans -6.88% to $+12.21\%$. During those runs, absolute combined throughput fell from about 978 kRPS to 753 kRPS and later recovered above 950 kRPS. The first 256 MiB/s pair shows a similar recovery. These shifts are much larger than the paper's $+0.44\%$ effect and track run time more plausibly than release rate.

With THP active, the release-off comparison lands in the paper's small-effect regime, but the run-to-run spread is larger than the median effect. Corrected release-on is now measured, but host drift prevents a stable rate-dependent conclusion.

6 Discussion and Related Work

At the methodology level, the Redis experiment was reconstructed locally as far as the public artifact allowed: the allocator split had to be rebuilt from public code, preload had to be verified at runtime, the compiler path had to be documented, and THP activity had to be observed rather than assumed.

The release-off result is consistent with the mechanism claimed in the paper. A recurring positive signal in the same sub-one-percent range suggests that the Redis effect is not obviously limited to Google's production environment. However, the result still comes

from a consumer i7-10700K host under Docker and WSL2, not from the paper's Skylake Xeon server environment.

The corrected release-on matrix does not reproduce a stable +0.44% effect. Its strongest apparent improvements occur in pairs where absolute host throughput recovered between consecutive allocator runs. Shorter interleaved blocks, CPU isolation, and a native Linux host would reduce this confounding. Future runs should also collect `/proc/vmstat` hugepage counters, load average, CPU frequency, and I/O activity throughout the benchmark.

The main limitation remains environmental fidelity. The paper specifies the Redis version, benchmark shape, CPU family, LLVM commit, and optimization level, but not the exact Linux distribution, kernel version, or THP policy for the Redis experiment. Docker, WSL2, consumer hardware, physical memory fragmentation, NUMA behavior, and hardware counter semantics can all affect hugepage experiments. The public TCMalloc commit is also only a historical approximation of the paper's internal allocator artifact.

Temeraire sits alongside other memory-management work on huge pages. SuperMalloc uses huge pages for very large allocations but does not make the general pageheap layout hugepage-aware [3]. MallocPool improves TLB behavior by serving objects from a contiguous preallocated region, but it is not a general-purpose pageheap replacement [2]. LLAMA uses lifetime prediction to group allocations that are likely to die together, whereas Temeraire uses online placement heuristics without profiling [5]. Kernel-level systems such as Ingens and HawkEye manage huge pages from the OS side and are complementary to allocator-side placement [4, 6].

7 Conclusion

This report should be read primarily as a reproduction of the Redis artefact. The original repository was not yet a Temeraire Redis reproduction. It had to be narrowed to the public Redis case study and adapted to Redis 6.0.9. It also had to shift from glibc and gperftools comparisons to legacy TCMalloc versus Temeraire, with historical TCMalloc builds, wrapper libraries, preload checks, compiler metadata, THP controls, and memory instrumentation.

After those changes, the release-off Redis result falls into the same small-effect regime as the paper: a local median of +0.48% against the paper's +0.75%, with substantial variation across the five run pairs. The corrected release-on matrix verifies that periodic release is active, but its rate medians do not form a stable trend and several pairs are dominated by host drift. The artifact reconstructs the public Redis path and documents where local measurement fidelity becomes the limiting factor. It does not validate Google's fleet-scale claims or reproduce the exact production-era measurements.

References

- [1] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. <https://www.usenix.org/conference/osdi21/presentation/hunter>
- [2] Marcus Jägemar. 2018. Mallocpool: Improving Memory Performance Through Contiguously TLB Mapped Memory. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1127–1130. doi:10.1109/ETFA.2018.8502619
- [3] Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management* (Portland, OR, USA) (*ISMM '15*). Association for Computing Machinery, New York, NY, USA, 41–55. doi:10.1145/2754169.2754178
- [4] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and efficient huge page management with ingens. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) (*OSDI'16*). USENIX Association, USA, 705–721.
- [5] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (*ASPLOS '20*). Association for Computing Machinery, New York, NY, USA, 541–556. doi:10.1145/3373376.3378525
- [6] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (*ASPLOS '19*). Association for Computing Machinery, New York, NY, USA, 347–360. doi:10.1145/3297858.3304064